# aculator

**Team Django Unchained**

Ramiz Özdemir, Esin Sarıaltın, Yaren Sargın, Rana İlgi Yılmaz

https://github.com/rmz-oz/aculator

Mid semester project report

Acıbadem University, Fall 2026

## CSE 311 Software Engineering

### Section 1 Punchline, key ingredient and backlog

**Punchline.** Every grade a department already enters becomes evidence for its eleven program outcomes (PO), and every PO score can be traced back to the grades behind it.

**Key ingredient.** Two layers of weighted averages, assessment to learning outcome (LO) and LO to PO, computed with exact decimals and rounded only on screen. The guideline example gives Elif PO 1 = 84.952, shown as 85.0; rounding LO 2 first, as the brief does, gives 84. Both numbers are unit tests.

**Backlog tool.** GitHub Issues: 53 issues so far, each from a task or bug template with an owner, a sprint milestone, an area label and a story point label. The first backlog came from the guidelines and the brief at the planning meeting on 29 October. The board is GitHub Projects, kept by the product owner. Planning and review meet every Thursday; points and notes go to `docs/sprints.md`, design questions to GitHub Discussions, and every change goes through a pull request with one approving review.

### Section 2 Division of work, difficulty, sprints and velocity

| Member, role | Sprint 1 (done) | Sprint 2 (done) |
|---|---|---|
| **Ramiz Özdemir** team lead, architecture | Django on PostgreSQL, CI, deploy to Render, architecture diagram | Roles and permissions, class diagram from the models, Getting Real notes, mid semester slides and report (merged) |
| **Esin Sarıaltın** backend, calculation | Core models with constraints and tests, weighting decision | LO and PO calculation service, PO components (merged) |
| **Yaren Sargın** data engineer | ER diagram, reproducible fake data command | Materialized views for student, course and cross course scores (merged) |
| **Rana İlgi Yılmaz** product owner, interface | Screen sketches, sprint notes | Course report page, project board and sprint notes (merged) |

**Difficulty.** Story points 1, 2, 3, 5 and 8, agreed in planning; an 8 means the issue should be split.

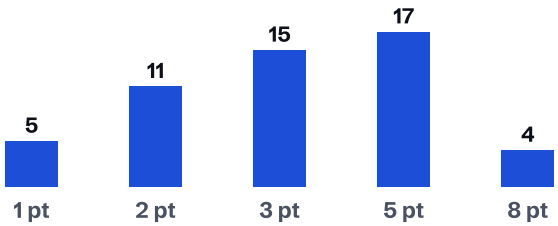



Figure 1. Issues per story point value.

**Sprints.** One week, Thursday to Thursday, eight sprints until 24 December. Completed: two.

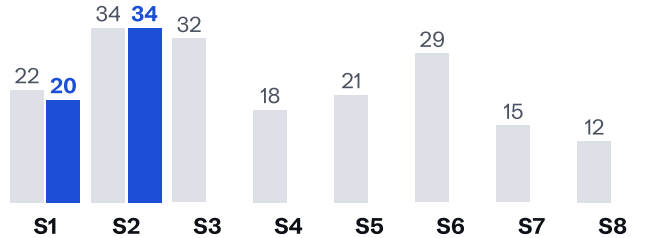



Figure 2. Story points planned (grey) and done (blue) per sprint.

**Velocity.** Sprint 1 (29 Oct to 5 Nov) finished 20 of 22 points; the unfinished board moved to Sprint 2. Sprint 2 (5 to 12 Nov) finished 34 of 34 points. Average velocity: 27 points per week.

# CSE 321 Data Systems

## Section 1 Introduction

aculator turns course grades into program outcome scores for a department, Acıbadem CSE first. Key features:

- **Two step calculation.** Assessment to LO with shares from 0 to 1, LO to PO component with levels from 1 to 5, across courses by ECTS; `NUMERIC` and `Decimal` throughout, rounded only for display.
- **Traceable scores.** Every PO score splits into what each assessment contributed (85.0 = 44.6 from the midterm + 40.4 from the project).
- **Terms kept apart.** LOs and assessments can change between terms; old reports stay intact.
- **Evidence per component.** As MÜDEK asks, a PO is met only when every one of its components is.
- **Three roles.** Students see their own scores, instructors their offerings, the head everything.

## Section 2 System design

**Architecture.** One Django 5.2 app, rendered on the server with HTMX, on PostgreSQL 16; both run on Render in Frankfurt. The app refreshes three materialized views after each grade change. Planned around them: a Redis cache that also queues bulk imports for a worker; university mail over SMTP for score changes and weak outcomes; grade import from ALMS (Moodle). GitHub Actions tests every pull request, builds an image to GHCR and deploys each merge to Render. One server rendered app means one language, one repository and one deploy.

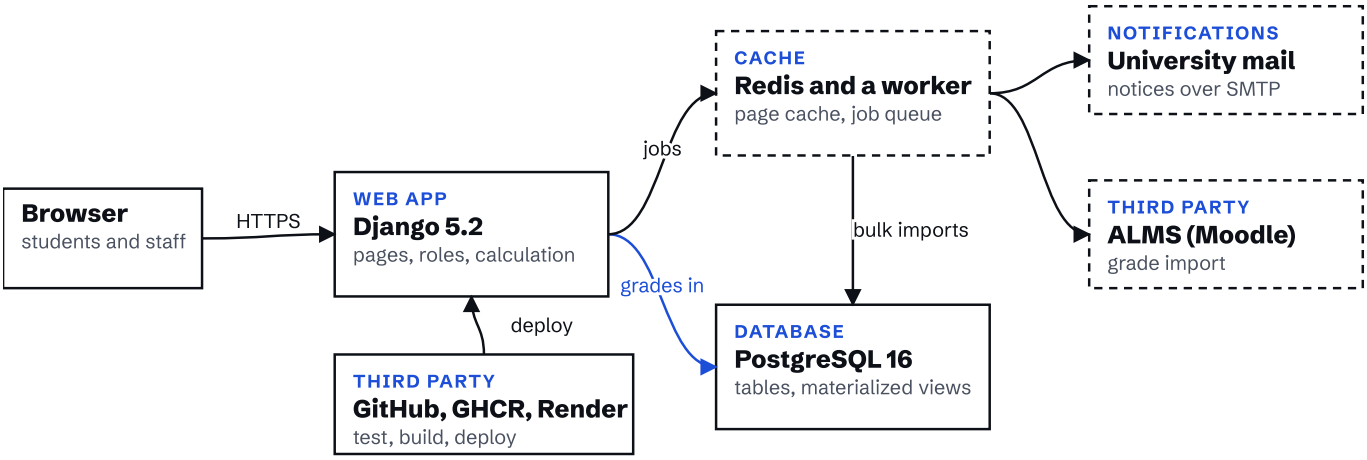



Figure 3. Architecture. Solid boxes run today, dashed boxes are planned; blue is the path of a grade.

**Classes and relationships.** Each class is one table. Course and Term make a CourseOffering, one per course per term. LearningOutcome, Assessment and Enrollment each belong to one offering; a User enrolls as a student, and instructors are linked to the offerings they teach, many to many. A Grade joins one enrollment and one assessment, with 0 ≤ score ≤ max_score. AssessmentLOWeight and LOPOWeight resolve the many to many links between assessments, LOs and PO components.

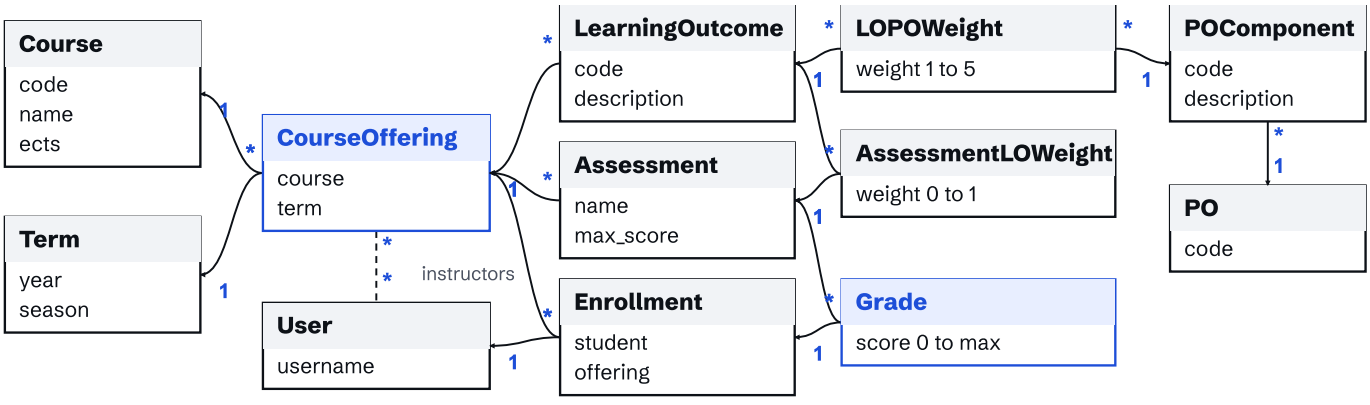



Figure 4. Classes. Arrows point to the referenced class; * many, 1 one; dashed, many to many.